

TD 2

Exercice 1

1. Écrire une fonction `range` telle que `range i j` renvoie `[i; i+1 ; ...; j]` , ou `[]` si $j < i$.
2. Ecrire une fonction `position` telle que `position x l` renvoie la première position à laquelle apparaît la valeur `x` dans la liste `l` . La fonction lève l'exception `Not_found` si `x` n'appartient pas à `l` .
3. Écrire une fonction qui rend la liste où tous les éléments de liste en argument sont dupliqués.
4. Écrire une fonction qui renvoie la liste des éléments de `l` dans l'ordre inverse.
5. Écrire une fonction qui donne le dernier élément d'une liste non vide et lève une exception sinon.
6. Écrire une fonction qui détermine si une liste contient un nombre pair d'éléments.
7. Écrire une fonction qui renvoie une liste privée de son dernier élément.
8. Écrire une fonction qui renvoie les `n` premiers éléments d'une liste.
9. Écrire une fonction qui détermine si une liste est un palindrome.
10. Écrire une fonction donnant le deuxième plus grand élément d'une liste.

Exercice 2

Nous souhaitons construire un dictionnaire afin de pouvoir vérifier l'orthographe d'un texte. Pour cela nous cherchons une représentation d'un dictionnaire. Un dictionnaire peut contenir par exemple les mots suivants : `art`, `article`, `artifice`, `balle`, `ballon`, `la`, `langage`, `langue`. Une solution naïve est de représenter ce dictionnaire par la séquence de ses mots : `["art" ; "article" ; "artifice" ; "balle" ; "ballon" ; "la" ; "langage" ; "langue"]`

1. Définir le type d'une lettre, d'un mot et d'un dictionnaire
2. Écrire une fonction `nbmots` qui compte le nombre de mots présents dans un dictionnaire.
3. Écrire une fonction `taillemot` qui compte le nombre de lettre d'un mot.
4. Écrire une fonction `nbkmots` qui compte le nombre de mots de taille `k` présents dans un dictionnaire.
5. Écrire une fonction `ajoutmot` qui ajoute un mot dans un dictionnaire.
6. Écrire une fonction `supprimemot` qui enlève un mot d'un dictionnaire.

Exercice 3

On va représenter un tableau d'effectifs de la manière suivante : si notre série statistique est représentée par une liste de type 'a list , notre tableau d'effectifs sera représenté par le type ('a * int) list , dans laquelle nous retrouveront toutes les valeurs possibles rangées dans l'ordre croissant, couplées avec leur effectif. Exemple :

```
let serie = [4; 5; 6; 4; 5; 4; 3; 1; 5; 6; 4] ;;  
let tab = [(1,1); (3,1); (4,4); (5,3); (6,2)] ;;
```

1. Ecrire une fonction ajoute_valeur : 'a -> ('a * int) list -> ('a * int) list qui ajoute une valeur dans un tableau d'effectifs.
2. Ecrire une fonction eff_tab : 'a list -> ('a * int) list * int qui prend en entrée une série statistique et qui renvoie le tableau d'effectifs associé ainsi que l'effectif total de la série.
3. Ecrire une fonction mediane : 'a list -> 'a qui prend en entrée une série statistique et qui en renvoie la médiane.